

Chapter 2

Computability

Procedure:

- ◆ **A procedure for solving a problem is a finite sequence of instructions which can be mechanically carried out given any input.**

- ◆ **Effective Procedure**
 - **Finitely describable**
 - **Well defined, discrete steps**
 - **Always terminates**

Algorithm

- ◆ AI algorithm is a procedure that terminates after a finite number of steps.

Limitations of computer science

Major reasons useful calculations cannot be done:

- ◆ Execution time of program is too long
 - problem can take years or centuries to finish
- ◆ Problem is not computable
 - no computer program can solve the problem
- ◆ We do not know how to write a program to solve the problem
 - problems that can conceivably be solved
 - includes many problems in artificial intelligence and computer vision, such as understanding language, object recognition, tracking, prediction

What problems can be solved by computers?

- ◆ Equivalent to: “What decision problems can be solved?”
- ◆ **Decision problem**
 - problem where the answer is always YES/NO
- ◆ Arbitrary problem can always be reduced to a decision problem
- ◆ There are decision problems for which algorithms do not exist
 - there are more different decision problems than there are different programs
 - if there are more decision problems than programs, then it must be the case that some decision problems cannot be solved by programs

Function evaluation

- ◆ A decision problem is a question with a YES/NO answer:
 - Is 613511 a prime number?
 - Is my birthday on a Sunday next year?
 - Did I pass the CPS1 final exam?
- ◆ Sometimes a particular answer from a set of possible solutions is required
 - What is the smallest prime factor of 613511?
 - On what day of the week is my birthday next year?
 - What grade will I get on the CPS1 final exam?
- ◆ The process of answering questions where a particular answer is uniquely determined from the given information can be viewed as the **evaluation of a function**

Functions

- ◆ A **function** is a correspondence between a collection of possible *input values* and a collection of *output values* such that each possible input is assigned a unique output
- ◆ Example: the addition function
 - inputs are value pairs, outputs are values representing the sum of each input pair
 - output is unique, exactly one value for each input pair
- ◆ Process of determining output for given input is called **computing** or **evaluating** the function
- ◆ The ability to evaluate functions gives us the ability to solve problems
 - to solve the addition problem we must evaluate the addition function

Computable functions

- **Algorithms** are the means by which we formally express the evaluation of a function
- Can we design algorithms to evaluate all functions?
 - No!
- Exist functions so complex that there is no well-defined step-by-step process for determining their output based on their input values
- The evaluation of these functions lies beyond the capabilities of any algorithmic system
- These functions are said to be **noncomputable**
- Functions whose output values can be determined algorithmically from their input values are said to be **computable**
 - functions having an algorithm that solves them

Limitations of machines

- Machines can only perform **computations** described by algorithms
 - the study of computable functions is the study of the **ultimate capabilities of machines**
- If we can build a machine with the capabilities to evaluate the entire set of computable functions, then the machines we build are as powerful as we can make them
- On the other hand ...
 - if we discover that the solutions to a problem requires the evaluation of a noncomputable function, we know the solution to that problem lies beyond the capabilities of machines

Is a problem computable?

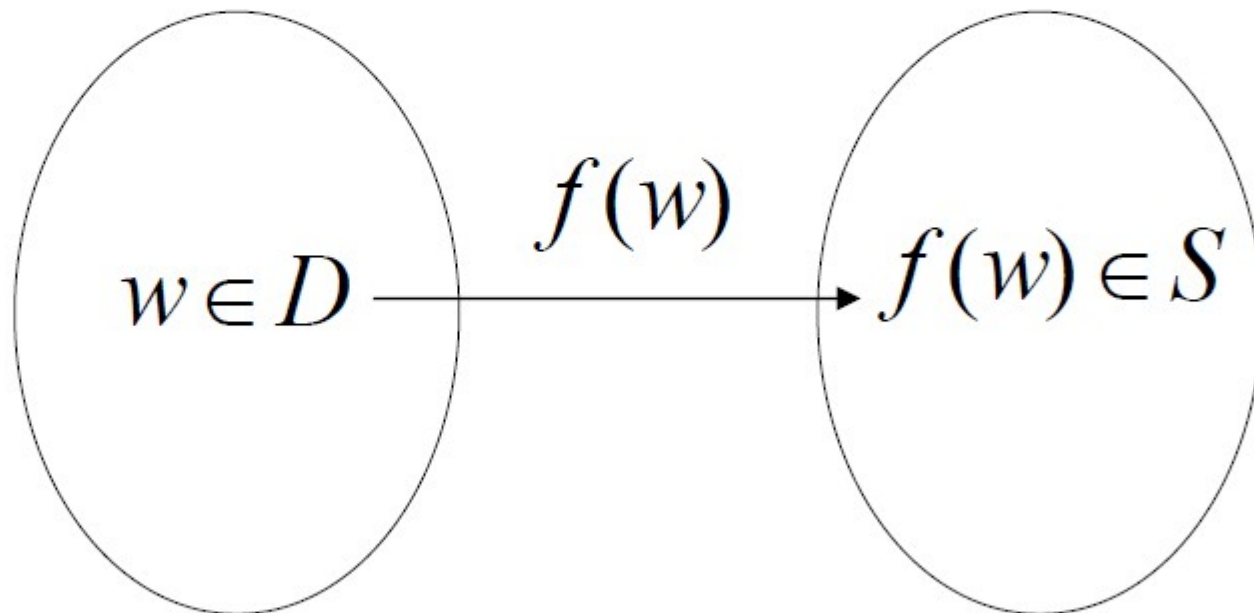
- ◆ Suppose we create a list of all programs for evaluating computable problems
- ◆ To ask whether a problem is computable is to ask if there is a program in the list to solve the problem
- ◆ This is not the same as actually identifying the program in the list that solves the problem
- ◆ There are many computable problems whose solutions have not yet been identified

Computing Functions with Turing Machines

A function $f(w)$ has:

Domain: D

Result Region: S



A function may have many parameters:

Example: Addition function

$$f(x, y) = x + y$$

Integer Domain

Decimal: 5

Binary: 101

Unary: 11111

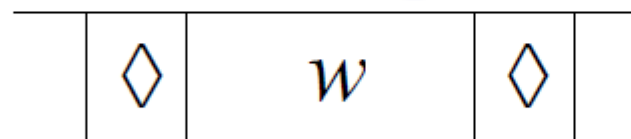
We prefer **unary** representation:

easier to manipulate with Turing machines

Definition:

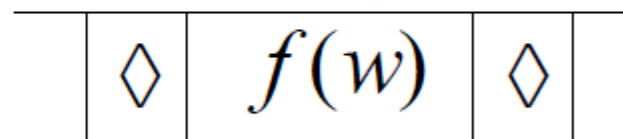
A function f is computable if
there is a Turing Machine M such that:

Initial configuration



q_0 initial state

Final configuration



q_f final state

For all $w \in D$ Domain

In other words:

A function f is computable if
there is a Turing Machine M such that:

$$q_0 w \xrightarrow{*} q_f f(w)$$

Initial
Configuration

Final
Configuration

For all $w \in D$ Domain

Turing computable

- ◆ Functions that can be computed by a Turing machine are said to be Turing computable

Turing-computable computable

- ◆ **A function is computable if it can be computed by a Turing machine**
 - or, a function is computable if a program exists that can compute it
 - or, a function is computable if there is an **effective algorithm** for evaluating the function which, given any input values for which the function is defined, *produces the correct result and halts in a finite amount of time*
- ◆ **Q:** Is there anything a Turing machine cannot do?
- ◆ **A:** Yes!

Example

The function $f(x, y) = x + y$ is computable

x, y are integers

Turing Machine:

Input string: $x0y$ unary

Output string: $xy0$ unary

Another Example

The function $f(x) = 2x$ is computable

x is integer

Turing Machine:

Input string: x unary

Output string: xx unary

Turing Machine Pseudocode for $f(x) = 2x$

1. Replace every 1 with \$
2. Repeat:
 3. Find rightmost \$, replace it with 1
 4. Go to right end, insert 1
5. Until no more \$ remain

Another Example

The function $f(x, y) = \begin{cases} 1 & \text{if } x > y \\ 0 & \text{if } x \leq y \end{cases}$ is computable

Turing Machine Pseudocode:

- **Repeat**

Match a 1 from x with a 1 from y

Until all of x or y is matched

- If a 1 from is not matched

erase tape, write 1 ($X > y$)

else

erase tape, write 0 ($x \leq y$)

Definition of Algorithm:

An algorithm for function $f(w)$

is a Turing Machine which computes $f(w)$

Algorithms are Turing Machines

When we say:

There exists an algorithm

We mean:

There exists a Turing Machine that executes the algorithm

Noncomputability

- ◆ A function is **noncomputable** if there exists no Turing machine that could compute it
- ◆ or, a noncomputable function is any function that cannot be computed by any program
 - many more functions than there are programs
 - not possible to have a program for every function

A noncomputable problem

- ◆ Problem: List all subroutines that input an integer and return an integer

- here is one short subroutine:

```
int sub1(int x)
{
    return 1;
}
```

- and here is another:

```
int sub2(int x)
{
    return 2;
}
```

- ◆ Can make infinitely many such subroutines

Halting

```
int programA( int x )
{
    while( x == x )
        x = x;
    return x;
}
```

```
int programB( int x )
{
    while( x > 10 )
        x = x;
    return x;
}
```

- ◆ Programs that run forever do not halt
 - **programA**
 - programs with infinite loops
- ◆ Programs may halt on some inputs but not on others
 - **programB**
- ◆ Programs with no loops always halt
 - programs composed of assignment statements, **if** statements and the **return** statement

Halt or not?

- ◆ Will the following code eventually terminate?

```
while (x > 1)
{
    if (x > 2)
        x = x - 2;
    else
        x = x + 2;
}
```

- ◆ What if x is 8? How about 9?

Halt or not?

- ◆ Will the following code eventually terminate?

```
while (x > 1)
{
    if ( x % 2 == 0 )
        x = x / 2;
    else
        x = 3*x + 1;
}
```

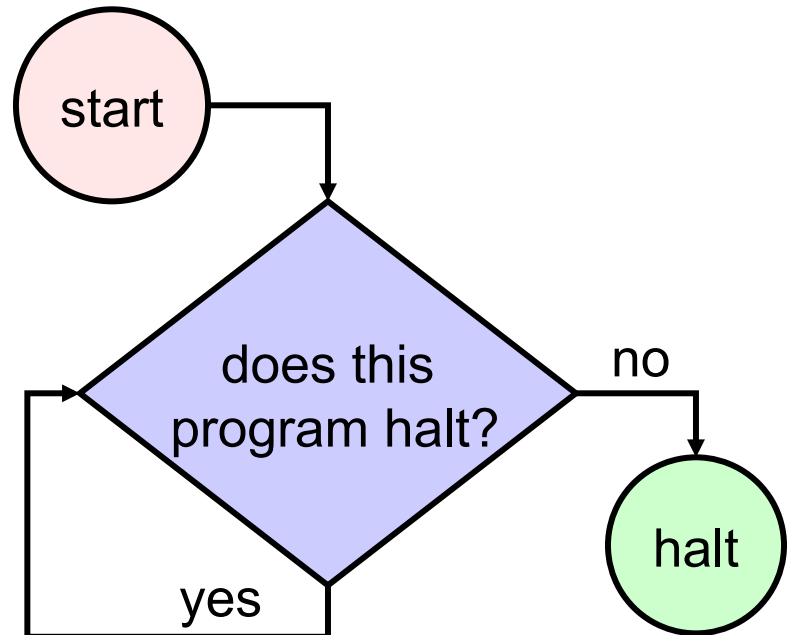
- ◆ What if **x** is 8? How about 7? How about any number > 0?

The halting problem

- ◆ **Q:** Is it possible to write a Java program to answer the following question

Does a given program halt on all inputs?

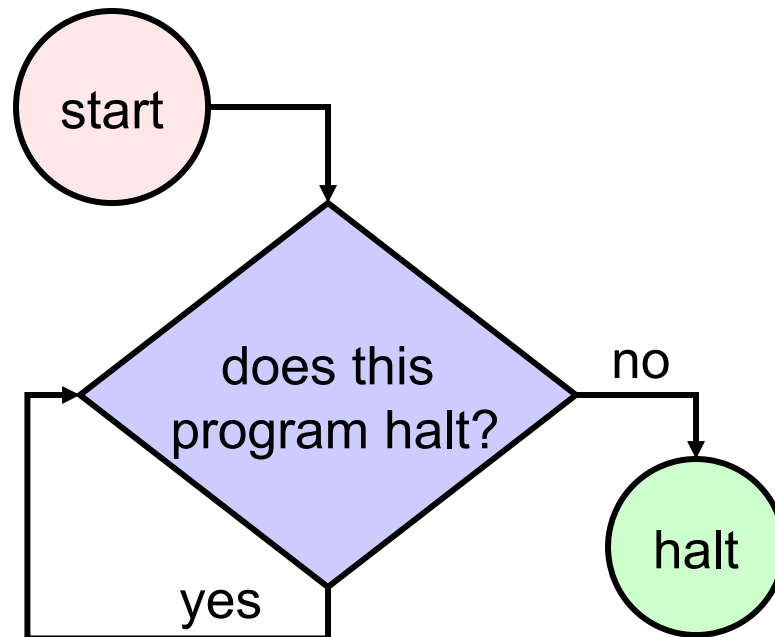
- ◆ It would be useful if we could write a program or compiler to test if a given source program contains an infinite loop
- ◆ If we had such a program, then we would never execute a program with an infinite loop
- ◆ This problem is known as the **halting problem**



The halting problem is noncomputable

No finite program can be written that will check other programs and halt in a finite time giving a solution to the halting problem

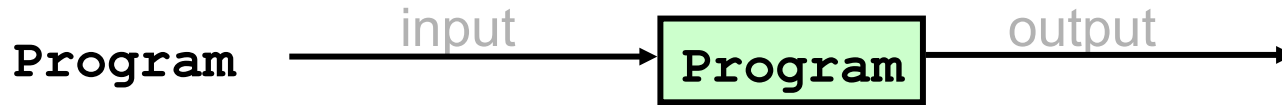
- ◆ There is no single finite program that will answer the halting question for all programs



Proof by contradiction

- ◆ How did Turing prove that such a program is theoretically impossible?
- ◆ Proof by contradiction:
 - Assume there is an algorithm **DoesHalt**(**P**, **I**) that returns “yes” if program **P** halts when given input **I** and “no” otherwise
 - Show that this assumption leads to a contradictory situation
 - Conclude that our initial assumption – the existence of a program to tell if another program halts – must be false

Programs that read programs



- ◆ Almost every problem related to the **behavior** of programs is noncomputable
 - programs to check for property X in the behavior of all other programs
 - halting, equivalence, printing, correctness
- ◆ We can write programs to check almost any **syntactic** feature of programs
 - a *compiler* is a program that reads programs
 - we can measure the length of programs, number of statements, characters, arithmetic expressions

Computable or noncomputable?

1. Write a program that inputs a sequence of characters and returns true if it is a legal Java program
2. Write a program that inputs a sequence of characters and returns true if any permutation of these characters is a legal Java program
3. Write a program that inputs a program in any programming language and returns true if the characters can be rearranged to form a legal Java program
4. Write a program that inputs a Java program and returns true if the program computes the sum of its inputs
5. Write a program that inputs two Java programs and returns true if both programs always produce the same output

Computable or noncomputable?

1. Computable
2. Computable
3. Computable
4. Noncomputable
5. Noncomputable

- ◆ **Recursively Enumerable Languages**
- ◆ **Recursive Language (REC)**
- ◆ **Recursively Enumerable (RE):**
- ◆ An RE language can be accepted or recognized by Turing machine. Which means it will enter into final state for the strings of language and may or may not enter into rejecting state for the strings which are not part of the language.

- ◆ **Recursive Enumerable (RE)...**
- ◆ It means TM can loop forever for the strings which are not a part of the language. RE languages are also called as **Turing recognizable languages.**
- ◆ **Let L be a RE language and M be the TM that accepts it.**
- ◆ **If** $w \in L$, then M halts in final state
- ◆ **If** $w \notin L$, the M halts in non-final state or loops forever.

- ◆ A TM may divide all strings on Σ^* into 3 states:
 - All accepted states
 - All strings on which TM rejects
 - All strings on which TM loops

- ◆ **Recursive Language (REC)**

- ◆ A recursive language (subset of RE) can be **decided** by **Turing machine** which means it will enter into final state for the strings of language and rejecting state for the strings which are not part of the language.

If L is a recursive language, there is a TM M such that:

- ◆ if $w \in L$, then M halts in a final (or accept) state
- ◆ if $w \notin L$, then M halts in a non final (or non accept) state

e.g.;

$L = \{a^n b^n c^n | n \geq 1\}$ is **recursive** because we can construct a turing machine which will move to final state if the string is of the form $a^n b^n c^n$ else move to non-final state.

◆ Recursive Language (REC)

So the TM will always halt in this case. REC languages are also called as Turing decidable languages. The relationship between RE and REC languages can be shown in

The set of Recursive languages is the subset of all RE.

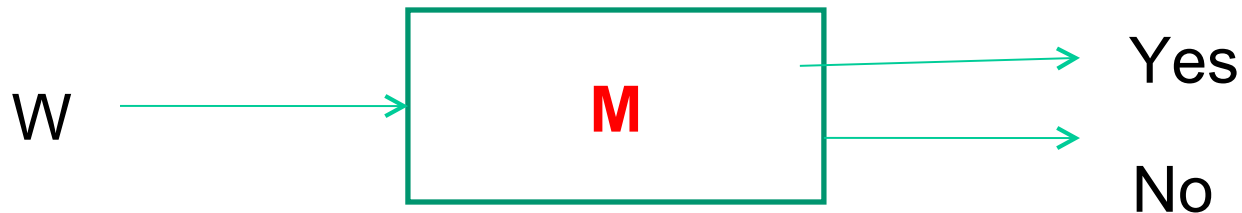


Figure 1

Closure Properties of Recursive Languages

TM Block Diagrams

If L is a recursive language, then a TM M that accepts L and always halts can be pictorially represented by a “box” that has one input and two outputs.



Closure Properties of Recursive Languages

If L is a recursively enumerable language, then a TM M that accepts L can be represented by a “box” that has one input and one output.



Complement

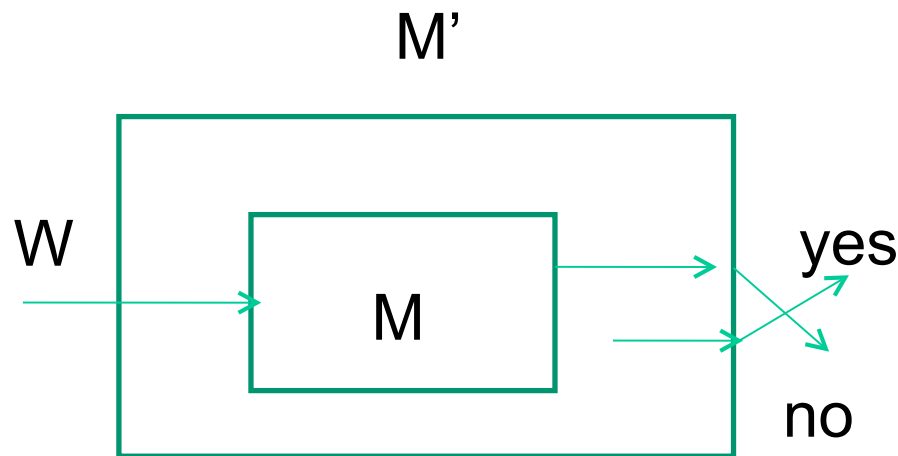
Theorem: The recursive languages are closed with respect to complementation.

♦ That is, if L is RL, then so L'

$$L' = \Sigma^* - L$$

Proof: Let M be a TM such that $L=L(M)$ and M always halts.

Construct TM M' as follows:



Note: M' accepts iff M does not

M' always halts since M always halts

From this, it follows that the complement of L is recursive.

Closure Properties of Recursive Languages

Theorem Union:

If L_1 and L_2 are two recursive languages, their union $L_1 \cup L_2$ will also be recursive.

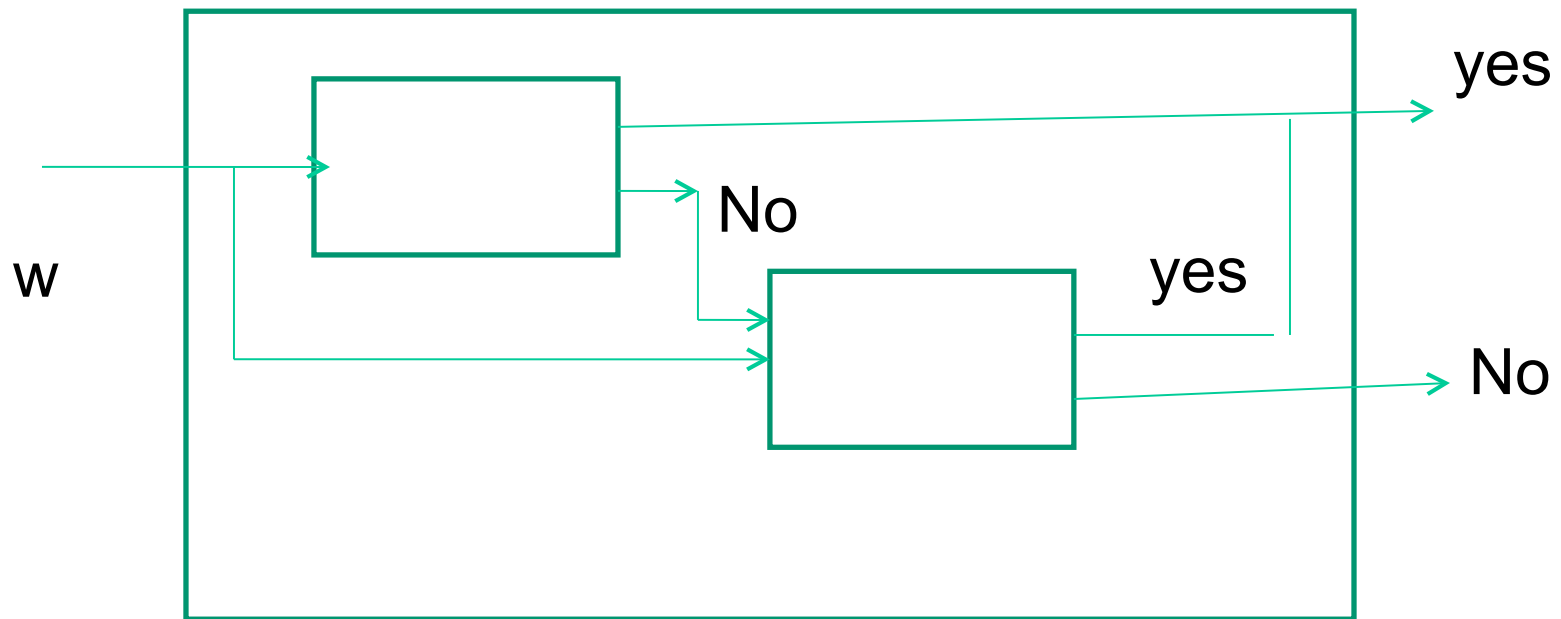
Proof:

Since L_1 and L_2 are recursive languages,



Note

- ◆ Let M_1 and M_2 be TMs such that $L_1 = L(M_1)$ and $L_2 = L(M_2)$ and M_1 and M_2 always halt. Construct TM M' as follows:



Note that: $L(M') = L(M_1) \cup L(M_2)$

M' always halts since M_1 and M_2 always halt.

Theorem :(Concatenation) If L_1 and L_2 are two recursive languages, their concatenation $L_1.L_2$ will also be recursive.
For Example:

$$L_1 = \{a^n b^n c^n | n \geq 0\}$$

$$L_2 = \{d^m e^m f^m | m \geq 0\}$$

$$L_3 = L_1.L_2 = \{a^n b^n c^n d^m e^m f^m | m \geq 0 \text{ and } n \geq 0\} \text{ is also recursive.}$$

Kleene Closure: If L_1 is recursive, its kleene closure L_1^* will also be recursive.

For Example:

$$L_1 = \{a^n b^n c^n \mid n \geq 0\}$$

$$L_1^* = \{a^n b^n c^n \mid n \geq 0\}^* \text{ is also recursive.}$$

Intersection

If L_1 and L_2 are two recursive languages, their intersection $L_1 \cap L_2$ will also be recursive.

For Example:

$$\begin{array}{ll} L_1 = \{a^n b^n c^n d^m & | \quad n \geq 0 \text{ and } m \geq 0\} \\ L_2 = \{a^n b^n c^n d^n & | \quad n \geq 0 \text{ and } m \geq 0\} \end{array}$$

$$\begin{aligned} L_3 &= L_1 \cap L_2 \\ &= \{a^n b^n c^n d^n \mid n \geq 0\} \text{ will be recursive.} \end{aligned}$$

◆ Closure Properties of Recursively Enumerable Languages

- ◆ If L , L_1 and L_2 are recursively enumerable languages, then so are

- $L_1 \cup L_2$,
- $L_1 L_2$,
- L^*

And

- $L_1 \cap L_2$

- ◆ Recursively enumerable languages are not closed under complement